

Relatorio - Padroes de Teste em Checkout de E-commerce

Disciplina: Engenharia de Software / Testes de Software

Trabalho: Padroes de Teste - Object Mother, Data Builder, Stubs e Mocks

Aluno: [preencher nome completo]

Matricula: [preencher matricula]

Repositorio: <https://github.com/CleitonSilvaT/test-pattern> ou fork do aluno

Este relatorio descreve a implementacao de uma suite de testes unitarios para o servico CheckoutService. O objetivo foi prevenir Test Smells por meio de padroes de criacao de dados de teste e test doubles, mantendo os testes legiveis, focados e isolados de dependencias externas.

1. Padroes de Criacao de Dados

1.1 Object Mother para User

O padrao Object Mother foi aplicado na classe UserMother para criar usuarios simples e recorrentes, como um usuario PADRAO e um usuario PREMIUM. Esse padrao e adequado quando a entidade tem poucos campos relevantes e quando seus cenarios mais comuns sao estaveis entre os testes.

1.2 Por que CarrinhoBuilder em vez de CarrinhoMother

O Carrinho e mais variavel que User: pode estar vazio, possuir diferentes usuarios e conter listas distintas de itens. Se fosse usado um CarrinhoMother, seriam necessarios muitos metodos como carrinhoVazio, carrinhoPremium, carrinhoComDoisItens e carrinhoComValorEspecifico. Essa proliferacao reduz a clareza e torna a manutencao mais dificil.

O CarrinhoBuilder resolve esse problema com uma API fluente. Ele fornece defaults seguros, mas permite explicitar apenas o que importa para cada caso de teste, como o usuario Premium e os itens que totalizam R\$ 200,00.

1.3 Antes: setup manual complexo

```
const user = new User(2, 'Usuario Premium', 'premium@email.com', 'PREMIUM');
const itens = [
  new Item('Produto A', 120),
  new Item('Produto B', 80),
];
const carrinho = new Carrinho(user, itens);
```

1.4 Depois: setup com Data Builder

```
const userPremium = UserMother.umUsuarioPremium();
const carrinho = new CarrinhoBuilder()
  .comUser(userPremium)
  .comItens([
    new Item('Produto A', 120),
    new Item('Produto B', 80),
  ])
  .build();
```

A segunda versao deixa a intencao do teste mais evidente: o que importa e que o carrinho pertence a um cliente Premium e possui R\$ 200,00 em itens. Os detalhes padrao ficam encapsulados no builder, reduzindo repeticao e setup obscuro.

2. Padroes de Test Doubles

2.1 Stub no cenario de falha de pagamento

No teste de falha, o GatewayPagamento foi usado como Stub. Sua funcao foi apenas controlar o fluxo do SUT retornando { success: false } quando cobrar fosse chamado. A verificacao principal foi de estado: o metodo processarPedido deveria retornar null e nao deveria persistir pedido nem enviar email.

```
const gatewayStub = {
  cobrar: jest.fn().mockResolvedValue({ success: false }),
};

const pedido = await checkoutService.processarPedido(carrinho, cartaoCredito);

expect(pedido).toBeNull();
```

2.2 Stub e Mock no cenario de sucesso Premium

No teste de sucesso para cliente Premium, o GatewayPagamento tambem atuou principalmente como Stub, pois retornou { success: true } para permitir que o fluxo continuasse. O PedidoRepository tambem foi usado como Stub, retornando um pedido salvo com id conhecido. Ja o EmailService foi usado como Mock, porque a finalidade do teste era verificar a interacao: se o email foi enviado uma vez e com os argumentos esperados.

```
expect(gatewayStub.cobrar).toHaveBeenCalledWith(180, cartaoCredito);
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  'premium@email.com',
  'Seu Pedido foi Aprovado!',
  'Pedido 10 no valor de R$180',
);
```

Essa separacao demonstra a diferenca entre verificacao de estado e verificacao de comportamento. O Stub ajuda a simular uma resposta externa e observar o resultado do SUT. O Mock, por sua vez, valida que uma colaboracao obrigatoria aconteceu da forma correta.

3. Conclusao

O uso deliberado de Object Mother, Data Builder, Stubs e Mocks torna a suite de testes mais sustentavel. Os builders reduzem setup obscuro e deixam cada teste expressar apenas o dado relevante para o cenario. Os test doubles isolam o CheckoutService de servicos externos, evitando testes frageis e lentos. Como resultado, a suite fica mais legivel, mais facil de manter e mais eficiente para detectar regressao nas regras de negocio.